2017 IEEE/ACM 39th IEEE International Conference on Software Engineering Companion

Mining Twitter Data for a More Responsive Software Engineering Process

Grant Williams* and Anas Mahmoud†
Division of Computer Science and Engineering, Louisiana Sate University
Baton Rouge, LA, 70803
*gwill83@lsu.edu,†mahmoud@csc.lsu.edu

***Abstract*—Twitter has created an unprecedented opportunity for software developers to monitor the opinions of large populations of end-users of their software. However, automatically classifying useful tweets is not a trivial task. Challenges stem from the scale of the data available, its unique format, diverse nature, and high percentage of spam. To overcome these challenges, this extended abstract introduces a three-fold procedure that is aimed at leveraging Twitter as a main source of technical feedback that software developers can benefit from. The main objective is to enable a more responsive, interactive, and adaptive software engineering process. Our analysis is conducted using a dataset of tweets collected from the Twitter feeds of three software systems. Our results provide an initial proof of the technical value of software-relevant tweets and uncover several challenges to be pursued in our future work.**

I. Introduction

Twitter has created an unprecedented opportunity for software providers to monitor the opinions of the end-users of their systems. Using Twitter, software users can publicly express their needs and concerns in the form of micro-blogs. Such data can be leveraged to extract rich and timely information about newly-released applications, enabling software developers to get instant technical and social feedback about their systems. However, information on Twitter is not always readily available. In particular, the quantity of Twitter data, its unique format and diverse nature, make the process of mining such data a challenging task [1], [2].

Motivated by these observations, in this extended abstract we ***a)*** conduct a manual qualitative analysis of a large sample of software-relevant Twitter data to determine the information value of software users' tweets, ***b)*** use text classification techniques to effectively capture and categorize the various types of actionable software maintenance requests present in such tweets, and ***c)*** investigate the performance of various text summarization techniques in generating compact summaries of the common technical concerns raised in software systems' Twitter feeds. Our main objective is to lay down an infrastructure for a more responsive and a more adaptive software engineering process that can achieve user satisfaction in an effective and a timely manner.

II. Preliminary Analysis

In this section, we describe the main steps of our research procedure and the experimental analysis associated with each step.

A. Data Collection and Qualitative Analysis

We used the Twitter *Search API* [3] to collect our dataset of software relevant tweets. In our analysis, we limit our data collection process to tweets addressed directly to the Twitter account of a given software system (e.g., tweets including *@Minecraft*). This strategy ensures that only tweets that are meant to be a direct interaction with the software provider are included. To conduct our analysis, we collected tweets from the Twitter feeds of three software systems, including: *Minecraft*, *Whatsapp*, and *Snapchat*. The data collection process extended from April 6th to June 4th of 2016, with duplicate tweets being discarded. The resulting dataset contained $51,792$ tweets. 400 tweets were then randomly sampled from the set of tweets collected for each of our systems. Sampled tweets were manually classified into three main categories, including: bug reports, feature requests, and others (e.g., spam and uninformative tweets) [4]. The data was classified by three human annotators with an average of 5 years of industrial software experience.

Table I describes our dataset, including our software systems, the total number of tweets, bug reports, feature requests, and other tweets identified for each system. Our qualitative manual analysis shows that, out of the 1200 tweets examined, 52% were technically informative (21% bug reports and 31% feature requests), while the other 48% were basically spam and miscellaneous information. These findings motivate the second and third phases of our analysis. In particular, given that around half of our data contains potentially useful information, how can that information be automatically classified and effectively summarized?

B. Automatic Classification

To automatically classify our data, we investigate the performance of two text classification algorithms, including Naive Bayes (NB) and Support Vector Machines (SVM). These two algorithms have been found to work well with short text. Short-text is a relatively recent Natural Language Processing (NLP) type of text that has been motivated by the explosive growth of micro-blogs on social media platforms (e.g., Twitter, YouTube, and Facebook) and the urgent need for effective methods to analyze such large amounts of limited textual data [2], [5].

To implement NB and SVM, we use Weka [6], a data mining software suite that implements a wide variety of machine

978-1-5386-1589-8/17 $31.00 © 2017 IEEE
DOI 10.1109/ICSE-C.2017.53

TABLE I
EXPERIMENTAL DATASET

System	Num. Tweets	Bugs	Features	Other
Minecraft	400	61	86	253
Whatsapp	400	112	118	170
Snapchat	400	80	172	148

learning and classification techniques. Stemming is applied to reduce words to their morphological roots. This leads to a reduction in the number of features (words) as only one base form of the word is considered. Recall and precision are used to evaluate the performance of our classifiers.

The performance of our classifiers is summarized in Table II. The results show that both NB and SVM have achieved competitive accuracy. In general, both classifiers were able to classify the different categories of tweets at decent levels of precision and recall. Closely examining our data instances shows that incorrectly classified instances of bug reports and feature requests commonly contained words of the nature *"please"* and *"help"* (e.g. *"please help me fix it"*). Such words tend to also appear frequently in irrelevant tweets of the nature *"please follow me on #snapchat"* or *"please help me reach a million followers"*, thus causing several of these tweets to be classified as irrelevant. A potential work-around for this problem is to consider phrases (combinations of uni-grams and bi-grams) rather than individual words in classification [7].

C. Summarization

The third phase of our analysis is focused on generating succinct summaries of the technically useful software tweets. The main objective is to assimilate the perspectives of a large number of end-users to focus software developers' attention on the most common concerns raised in their Twitter feeds [8]. In our analysis, we examine the performance of two frequency-based extractive summarization techniques that have been used heavily to summarize Twitter data [9], [10]. An extractive summary selects individual tweets from the set as representatives of the entire set. These techniques include:

- **Average Term Frequency (TF)**: The probability of a tweet to appear in a summary is correlated with the average TF weight of its words. The TF weight of a word is computed as its frequency in a collection of tweets divided by the total number of words in the collection.
- **SumBasic**: SumBasic uses the average term frequency (TF) of tweets' words to determine their value. However, the weight of individual words is updated after the selection of a tweet to minimize redundancy [9]. This approach can be described as follows:
  1) The probability of a word in the input corpus of size N words is calculated as $\rho(w_i) = n/N$, where n is the frequency of the word in the entire corpus.
  2) The weight of a tweet S_j is calculated as the average probability of its words, given by $\sum_{i=1}^{|S_j|} \rho(w_i)/|S_j|$
  3) The best scoring tweet is selected. For each word in the selected tweet, its probability is reduced by $\rho(w_i)_{new} = \rho(w_i) \times \rho(w_i)$.
  4) Repeat from 2 until the required length of the summary is met.

TABLE II
PERFORMANCE OF NB AND SVM IN CLASSIFYING TECHNICAL TWEETS

	NB		SVM	
Label	**Precision**	**Recall**	**Precision**	**Recall**
Bug Reports	0.81	0.74	0.76	0.78
Feature Requests	0.79	0.76	0.77	0.73

TABLE III
THE RECALL OF THE SUMMARIZATION TECHNIQUES

	Minecraft	*Whatsapp*	*Snapchat*
TF	0.41	0.31	0.44
SumBasic	0.68	0.70	0.72

To evaluate our summarization techniques, our human annotators were provided with the sets of bug report and feature request tweets for each system, and then were asked to select 10 tweets from each set that they thought were representatives of the main topics raised in the set. Average TF and SumBasic were applied to the data. The quality of generated summaries was then assessed as the percentage of term overlap (recall) between the human-generated summaries and the automated summaries after removing common English stop words and applying stemming. The results, shown in Table III, show that SumBasic, a simple frequency-based technique with redundancy control, was more successful than average TF in recalling concerns that our experts selected in their summaries.

III. CONCLUSIONS AND FUTURE WORK

In this extended abstract we introduced a preliminary multi-step procedure that is aimed at leveraging Twitter as a main source of technical information that software developers can benefit from. Our analysis was conducted using tweets sampled from the Twitter feeds of three software systems. These tweets were manually classified into different categories of software maintenance requests, including bug reports and feature requests. Multiple text classifiers were then used to automatically classify the data. The results showed that classifiers such as NB and SVM can be very effective in detecting different categories of technically-informative tweets. Our results also showed that SumBasic, a frequency-based summarization technique with redundancy control, can generate high quality summaries of software-relevant tweets.

The line of work in this paper has opened several research directions to be pursued in the future. For instance, a major part of our future effort will be devoted to collecting larger datasets, investigating the performance of more advanced Twitter data classification and summarization techniques, developing working prototypes that implement our main findings, and conducting human studies to demonstrate the practical significance of our findings and test the various usability aspects of our tools.

REFERENCES

[1] J. Bollen, H. Mao, and X. Zeng, "Twitter mood predicts the stock market," *Journal of Computational Science*, vol. 2, no. 1, pp. 1–8, 2011.

[2] M. Conover, B. Goncalves, J. Ratkiewicz, A. Flammini, and F. Mencze, "Political polarization on twitter," in *International AAAI Conference on Weblogs and Social Media*, 2011, pp. 89–96.

[3] "Twitter developer api," https://dev.twitter.com/, accessed: August, 15th.

[4] N. Chen, J. Lin, S. Hoi, X. Xiao, and B. Zhang, "Ar-miner: mining informative reviews for developers from mobile app marketplace," in *International Conference on Software Engineering*, 2014, pp. 767–778.

[5] M. McCord and M. Chuah, "Spam detection on twitter using traditional classifiers," in *international conference on Autonomic and trusted computing*, 2011, pp. 175–186.

[6] "Weka 3: Data mining software in java," http://www.cs.waikato.ac.nz/ ml/weka/, accessed: August, 15th.

[7] A. Pak and P. Paroubek, "Twitter as a corpus for sentiment analysis and opinion mining," in *Language Resources and Evaluation Conference*, 2010, pp. 1320–1326.

[8] C. Llewellyn, C. Grover, and J. Oberlander, "Summarizing newspaper comments," in *International Conference on Weblogs and Social Media*, 2014, pp. 599–602.

[9] A. Nenkova and L. Vanderwende, "The impact of frequency on summarization," Microsoft Research, Tech. Rep., 2005.

[10] D. Inouye and J. Kalita, "Comparing Twitter summarization algorithms for multiple post summaries," in *International Conference on Social Computing (SocialCom) and International Conference on Privacy, Security, Risk and Trust (PASSAT)*, 2011, pp. 298–306.